

TP Bonus — CI/CD avec GitHub Actions (version debutant)

Objectifs

A la fin de ce TP, vous saurez :

1. Créer un **repository GitHub** et y pousser du code
2. Ecrire un **workflow GitHub Actions** (equivalent du `.gitlab-ci.yml`)
3. Utiliser le **cache** pour acclereler les pipelines
4. Generer et telecharger des **artifacts**
5. Tester sur **plusieurs versions de Node.js** avec une matrice
6. Configurer un **deploiement staging** (automatique) et **production** (manuel)
7. Utiliser les **secrets GitHub** (equivalent des variables CI/CD de GitLab)
8. Comprendre les **différences** entre GitLab CI et GitHub Actions

Prerequis

- Avoir fait les TP1 a TP4 (ou au moins les avoir lus)
- Un **compte GitHub** (gratuit sur <https://github.com>)
- Git Bash installe sur votre machine
- Node.js 18+ installe (pour tester en local)

Etape 1 — GitLab CI vs GitHub Actions : les differences

Avant de commencer, comprenons les equivalences :

Concept	GitLab CI	GitHub Actions
Fichier de config	<code>.gitlab-ci.yml</code> (a la racine)	<code>.github/workflows/*.yml</code> (dans un dossier)
Declencheur	<code>workflow: rules:</code>	<code>on: push/pull_request</code>
Etape	<code>stages</code>	Pas de stages, on utilise <code>needs</code>
Job	Un bloc YAML nomme	Un bloc dans <code>jobs:</code>
Image Docker	<code>image: node:18-alpine</code>	<code>runs-on: ubuntu-latest</code> + <code>setup-node</code>
Cache	<code>cache: key: / paths:</code>	<code>actions/cache@v4</code>
Artifacts	<code>artifacts: paths:</code>	<code>actions/upload-artifact@v4</code>
Variables	<code>variables:</code>	<code>env:</code>
Secrets	Settings > CI/CD > Variables (masked)	Settings > Secrets > Actions
Deploy manuel	<code>when: manual</code>	<code>environment:</code> avec "Required reviewers"
Runner	GitLab Runner (shell/docker)	GitHub-hosted runners (ubuntu, windows, macos)

Clone du code	Automatique	Manuel avec `actions/checkout@v4`
---------------	-------------	-----------------------------------

****Difference importante**** : sur GitLab, le code est clone automatiquement dans chaque job. Sur GitHub Actions,

****Autre difference**** : GitLab CI utilise des ****stages**** (install → test → build → deploy) pour definir l'ordre

Etape 2 — Creer le repository sur GitHub

1. Allez sur <https://github.com>
2. Cliquez sur le + en haut a droite > **New repository**
3. **Repository name** : bonus-github-actions
4. **IMPORTANT** : **Ne cochez PAS** "Add a README file"
5. Cliquez sur **Create repository**
6. Gardez l'URL HTTPS de cote (ex:
<https://github.com/votre-user/bonus-github-actions.git>)

****Pourquoi ne pas cocher le README ?**** Meme raison que sur GitLab : si le repo distant a un fichier et que vot

Etape 3 — Preparer le code en local

Le code est fourni dans le dossier `bonus github action/`. Copiez-le dans un dossier de travail :

```
# Aller sur le Bureau
cd ~/Desktop/"CI CD"

# Creer un dossier de travail
mkdir bonus-work
cd bonus-work

# Copier les fichiers du projet
cp -r "../bonus github action/src" ./
cp -r "../bonus github action/tests" ./
cp -r "../bonus github action/scripts" ./
cp "../bonus github action/package.json" ./
cp "../bonus github action/.gitignore" ./

# Copier les workflows GitHub Actions
mkdir -p .github/workflows
cp "../bonus github action/.github/workflows/ci.yml" .github/workflows/
cp "../bonus github action/.github/workflows/test-matrix.yml" .github/workflows/
cp "../bonus github action/.github/workflows/deploy.yml" .github/workflows/
```

Verifier que tout est la :

```
ls -la
ls -la .github/workflows/
```

Vous devez voir :

- A la racine : .gitignore, package.json, src/, tests/, scripts/, .github/
- Dans .github/workflows/ : ci.yml, test-matrix.yml, deploy.yml

Tester en local :

```
npm install
npm test
```

Vous devez voir : Tests: 17 passed, 17 total. Puis :

```
rm -rf node_modules
```

****Pourquoi supprimer node_modules ?**** Parce que le .gitignore les ignore, mais c'est une bonne habitude de v

Etape 4 — Comprendre le Workflow 1 : CI Pipeline

Ouvrez le fichier .github/workflows/ci.yml dans VS Code :

```
code .github/workflows/ci.yml
```

Structure d'un workflow GitHub Actions :

```
name: CI Pipeline          # Nom affiche dans l'interface GitHub

on:                        # QUAND le workflow se declenche
  push:
    branches: [main]      # A chaque push sur main
  pull_request:
    branches: [main]      # A chaque PR vers main

env:                       # Variables globales
  NODE_VERSION: "18"

jobs:                      # Les jobs a executer
  install:                 # Nom du job
    runs-on: ubuntu-latest # Sur quel type de machine
    steps:                 # Les etapes du job
      - name: Etape 1      # Nom de l'etape
        run: echo "Hello"  # Commande a executer
```

Les 4 jobs du workflow CI :

Job	Depend de	Ce qu'il fait
`install`	-	Installe les dependances et cree le cache
`test-unit`	`install`	Lance les tests unitaires
`test-integration`	`install`	Lance les tests d'integration
`build`	`test-unit` + `test-integration`	Construit l'application

****Equivalent GitLab :****

- `install` = stage `install` avec `cache: policy: push`
- `test-unit` + `test-integration` = stage `test` avec `cache: policy: pull`
- `build` = stage `build` avec artifacts `dist/`

Focus : les Actions (steps reutilisables)

GitHub Actions a un concept unique : les **Actions**. Ce sont des etapes reutilisables creees par la communaute.

```
# ACTION : Cloner le code (obligatoire sur GitHub !)  
- uses: actions/checkout@v4  
  
# ACTION : Installer Node.js avec cache  
- uses: actions/setup-node@v4  
  with:  
    node-version: "18"  
    cache: "npm"  
  
# ACTION : Uploader un artifact  
- uses: actions/upload-artifact@v4  
  with:  
    name: mon-artifact  
    path: dist/
```

****C'est comme un plugin**** : au lieu d'ecrire 10 lignes de commandes bash, vous utilisez une Action en une ligne

Focus : le cache dans GitHub Actions

```
# SAUVEGARDER le cache (equivalent GitLab : policy: push)  
- uses: actions/cache/save@v4  
  with:  
    path: node_modules  
    key: node-modules-${{ hashFiles('package-lock.json') }}  
  
# RESTAURER le cache (equivalent GitLab : policy: pull)  
- uses: actions/cache/restore@v4  
  with:  
    path: node_modules  
    key: node-modules-${{ hashFiles('package-lock.json') }}
```

****`\${{ hashFiles('package-lock.json') }}`**** calcule un hash du fichier. Si le `package-lock.json` ne change pas

Focus : les artifacts

```
# UPLOADER un artifact (equivalent GitLab : artifacts: paths:)  
- uses: actions/upload-artifact@v4  
  with:  
    name: build-${{ github.sha }} # Nom unique avec le hash du commit  
    path: dist/ # Dossier a sauvegarder  
    retention-days: 7 # Duree de conservation (7 jours)
```

****Difference avec GitLab**** : sur GitLab, les artifacts sont integres dans le pipeline. Sur GitHub, on utilise

Etape 5 — Comprendre le Workflow 2 : Matrice de tests

Ouvrez le fichier `.github/workflows/test-matrix.yml` :

```
code .github/workflows/test-matrix.yml
```

La strategie matrice

```
strategy:
  matrix:
    node-version: [18, 20]
```

****Que fait cette matrice ?**** GitHub Actions cree ****automatiquement**** un job pour chaque valeur de la matrice.

- Un job avec ``node-version: 18``
- Un job avec ``node-version: 20``

Les deux jobs tournent ****en parallele****.

****Comparaison avec GitLab**** : dans le TP3, on devait ****dupliquer**** le job (``test-node-18`` et ``test-node-20``).

Le job runner-info

Ce job affiche les variables predefinies de GitHub Actions :

Variable GitHub Actions	Equivalent GitLab
<code>`github.sha`</code>	<code>`\$CI_COMMIT_SHA`</code>
<code>`github.ref_name`</code>	<code>`\$CI_COMMIT_BRANCH`</code>
<code>`github.run_id`</code>	<code>`\$CI_PIPELINE_ID`</code>
<code>`github.repository`</code>	<code>`\$CI_PROJECT_PATH`</code>
<code>`runner.os`</code>	<code>`\$CI_RUNNER_DESCRIPTION`</code>
<code>`github.actor`</code>	<code>`\$GITLAB_USER_LOGIN`</code>
<code>`github.event_name`</code>	<code>`\$CI_PIPELINE_SOURCE`</code>

Etape 6 — Comprendre le Workflow 3 : Deploiement

Ouvrez le fichier `.github/workflows/deploy.yml` :

```
code .github/workflows/deploy.yml
```

Declenchement conditionnel

```
on:
  workflow_run:
    workflows: ["CI Pipeline"]
    branches: [main]
    types: [completed]
```

****Que fait `workflow\_run` ?**** Ce workflow ne se déclenche que quand le workflow "CI Pipeline" ****termine**** sur

Les environments GitHub

```
environment:
  name: staging
  url: https://staging.example.com
```

****Les environments**** fonctionnent comme dans GitLab. Ils sont visibles dans l'onglet ****Environments**** de votre

- ****Required reviewers**** : quelqu'un doit approuver avant le déploiement (équivalent de `when: manual` dans Gi
- ****Wait timer**** : attendre X minutes avant le déploiement
- ****Branch restrictions**** : n'autoriser que certaines branches

Déploiement manuel (production)

Pour rendre le déploiement production **manuel** (comme `when: manual` dans GitLab) :

1. Allez dans votre repo GitHub > **Settings** > **Environments**
2. Cliquez sur **production** (créé automatiquement au premier run)
3. Cochez **Required reviewers**
4. Ajoutez votre nom d'utilisateur
5. Cliquez **Save protection rules**

Désormais, quand le workflow arrive au job `deploy-production`, il **attendra votre approbation** dans l'interface GitHub.

Etape 7 — Pousser vers GitHub et observer

```
# Initialiser Git
git init -b main

# Ajouter tous les fichiers
git add .

# Vérifier (pas de node_modules !)
git status

# Committer
git commit -m "init: bonus GitHub Actions CI/CD"

# Connecter au repository GitHub (REMPLACEZ L'URL)
git remote add origin https://github.com/votre-user/bonus-github-actions.git

# Pousser
git push -u origin main
```

****Des que vous poussez****, GitHub detecte automatiquement les fichiers dans `.github/workflows/` et lance les w

Etape 8 — Observer les workflows dans GitHub

8.1 — Voir les workflows

1. Allez sur votre repository GitHub
2. Cliquez sur l'onglet **Actions** (en haut)
3. Vous verrez les workflows en cours d'execution :
 - **CI Pipeline** : le workflow principal (install → test → build)
 - **Tests Multi-Versions** : les tests sur Node 18 et Node 20
4. Cliquez sur un workflow pour voir le detail

8.2 — Voir le detail d'un job

5. Cliquez sur un job (ex: Tests unitaires)
6. Vous voyez chaque **step** avec ses logs (comme dans GitLab)
7. Les steps verts = reussis, les steps rouges = echoues

8.3 — Comparer les versions de Node

8. Dans le workflow **Tests Multi-Versions**, cliquez sur Tests Node 18
9. Cherchez la ligne `Node.js version`
10. Faites de meme avec Tests Node 20
11. Les versions sont differentes (comme dans le TP3 avec l'executor Docker)

8.4 — Telecharger les artifacts

12. Sur la page du workflow **CI Pipeline**, scrollez en bas
13. Vous verrez la section **Artifacts** avec build-
14. Cliquez dessus pour telecharger le zip contenant dist/

8.5 — Voir les informations du Runner

15. Dans le workflow **Tests Multi-Versions**, cliquez sur Informations Runner
16. Vous verrez toutes les variables GitHub Actions (commit, branche, runner OS, etc.)

Etape 9 — Observer le cache en action

Premier push (cache froid)

Dans le job `install` du workflow **CI Pipeline**, vous verrez :

```
Cache not found for input keys: node-modules-abc123
```

C'est normal : le cache n'existe pas encore.

Deuxieme push (cache chaud)

Faites une petite modification et poussez :

```
echo "// test cache" >> src/utils.js
git add .
git commit -m "test: verifier le cache GitHub Actions"
git push
```

Dans le job test-unit, vous verrez :

```
Cache restored successfully
```

Le cache node_modules est reutilise et le job est plus rapide !

Etape 10 — Créer un secret GitHub (equivalent des variables GitLab)

10.1 — Créer le secret

1. Allez dans votre repo > **Settings** > **Secrets and variables** > **Actions**
2. Cliquez sur **New repository secret**
3. **Name** : DEPLOY_TOKEN
4. **Secret** : mon-token-secret-123
5. Cliquez **Add secret**

****Difference avec GitLab**** : sur GitLab, les variables peuvent etre "masked" (masquees dans les logs) ou "prot

10.2 — Utiliser le secret dans un workflow

Le secret est utilise dans le workflow deploy.yml :

```
- name: Utiliser un secret
  run: |
    echo "Le secret DEPLOY_TOKEN existe : ${ secrets.DEPLOY_TOKEN != '' }"
```

****REGLE ABSOLUE**** (meme que GitLab) : ne JAMAIS mettre de secrets dans le code ou les fichiers YAML. Utilisez

Etape 11 — Configurer le deploiement manuel (optionnel)

Pour que le deploiement en production necessite une approbation :

1. Allez dans **Settings > Environments**
2. Si l'environnement production n'existe pas, cliquez **New environment > production**
3. Cochez **Required reviewers**
4. Ajoutez votre nom d'utilisateur GitHub
5. Cliquez **Save protection rules**

Desormais, le job `deploy-production` attendra votre approbation. Vous recevrez une notification et pourrez approuver depuis l'onglet **Actions**.

Recapitulatif

Concept	GitLab CI	GitHub Actions
Fichier	<code>.gitlab-ci.yml</code>	<code>.github/workflows/*.yml</code>
Clone du code	Automatique	<code>actions/checkout@v4</code> (obligatoire)
Image Docker	<code>image: node:18</code>	<code>runs-on: ubuntu-latest</code> + <code>setup-node</code>
Ordre des jobs	<code>stages</code>	<code>needs</code>
Cache	<code>cache: key: / paths:</code>	<code>actions/cache@v4</code>
Artifacts	<code>artifacts: paths:</code>	<code>actions/upload-artifact@v4</code>
Matrice de tests	Dupliquer les jobs	<code>strategy: matrix:</code>
Variables	<code>variables:</code>	<code>env:</code>
Secrets	Settings > CI/CD > Variables (masked)	Settings > Secrets > Actions
Deploy manuel	<code>when: manual</code>	Environment + Required reviewers
Pipelines multiples	1 seul fichier <code>.gitlab-ci.yml</code>	Plusieurs fichiers dans <code>workflows/</code>
Minutes gratuites	400 min/mois	2000 min/mois (plus genereux !)

Erreurs courantes et solutions

Erreur	Cause	Solution
Workflow ne se declenche pas	Fichier pas dans <code>.github/workflows/</code>	Verifier le chemin exact (le <code>.</code> de <code>.github</code> est important)
<code>actions/checkout@v4</code> echoue	Permissions du token	Verifier Settings > Actions > General > Workflow permissions
Cache jamais restaure	Cle differente entre save et restore	Utiliser exactement la meme expression <code>hashFiles(...)</code>
Secret affiche <code>****</code>	C'est normal !	GitHub masque automatiquement les secrets dans les logs
Job <code>deploy</code> ne se declenche pas	<code>workflow_run</code> mal configure	Verifier que le nom du workflow reference est exact (sensible a la casse)
<code>Permission denied</code> au push	Authentification HTTPS	Utiliser un Personal Access Token ou configurer SSH

Pour aller plus loin

- **GitHub Actions Marketplace** : <https://github.com/marketplace?type=actions> (des milliers d'Actions pretes a l'emploi)
- **Self-hosted runners** : installer votre propre runner (comme les specific runners GitLab)
- **Docker dans GitHub Actions** : construire et pousser des images Docker vers GitHub Container Registry
- **Reusable workflows** : creer des workflows reutilisables (comme les include GitLab)
- **Branch protection rules** : proteger la branche main et exiger des checks verts avant le merge

Fin du TP Bonus. Vous savez maintenant utiliser les deux plateformes CI/CD les plus populaires : GitLab CI et GitHub Actions !